

Smart Marine — Project Report

Executive Summary

Smart Marine is an AI-driven computer vision system that detects plastic waste in marine environments from images and video (drone footage, webcams). The project is designed to be **auditable, testable, benchmarked, and deployment-ready** for a professional demo and as a foundation for production hardening.

Key strengths:

- **Traceability:** run metadata (`run_id`, `model_version`, timestamps) and structured logs.
- **Reliability:** retry/skip policy for failed frames and low-confidence warnings.
- **Benchmarking:** FPS + latency percentiles + CPU/RSS metrics, including stress mode.
- **Deployment:** Docker-based runtime for reproducibility.
- **QA:** automated pytest suite + CI-friendly behavior.

Key remaining real-world checks (outside a demo environment):

- GPU / optimized runtime validation (CUDA / ONNX / TensorRT) for real-time 30+ FPS targets.
 - multi-hour flight testing on representative drone payloads.
-

1. Project Purpose

1.1 Problem

Manual inspection of drone or camera footage for marine plastic waste is slow and inconsistent. Operators need assistance to surface candidate frames/regions quickly.

1.2 Goals

- Detect plastic waste in **real-time** (where feasible) or **batch** workflows.
 - Provide results that are **repeatable and auditable**.
 - Support professional engineering expectations:
 - error handling
 - observability (logs + metrics)
 - tests
 - deployment guidance
-

2. Tech Stack

2.1 Core

- Python 3.x
- PyTorch (YOLOv5 inference)
- OpenCV (video capture/decoding)
- FastAPI (REST API)
- Docker (reproducible runtime)
- pytest (automated tests)

2.2 Benchmarking & Observability

- `logging` (structured logs)
- `psutil` (CPU/RSS capture)

2.3 Optional / Future Optimization

- ONNX export + ONNX Runtime
 - TensorRT (NVIDIA)
-

3. Architecture Overview

Smart Marine is composed of three primary layers:

1. **Inference Core (PlasticDetector)**
2. **API Layer (FastAPI server)**
3. **Tooling Layer (benchmarking, QA tests, docs)**

3.1 High-level Flow

1. Input arrives as:
 - image file
 - directory of images
 - video file
 - webcam frames
 2. Frames are processed by the detector.
 3. The system emits:
 - per-frame detections
 - run metadata
 - warnings
 - batch summary
 4. Logs and benchmark reports provide operational visibility.
-

4. Core Component: PlasticDetector

4.1 Responsibilities

- load model weights
- run inference
- render annotated output
- compute batch summaries
- emit warnings and reliability signals

4.2 Key Features

4.2.1 Traceability

Each run includes:

- `run_id`: unique UUID
- `model_version`: derived from model weights for reproducibility
- `timestamp`
- config snapshot (thresholds, device, image size)

4.2.2 Reliability / Robustness

- **Retry policy:** `max_retries` for batch processing of images.
- **Skip on failure:** if a file continues failing after retries, it is skipped and counted in the summary.

4.2.3 Low-confidence warnings

- `low_conf_warning_threshold` flags uncertain detections.
- Per-image warnings are attached to outputs.
- Batch summaries aggregate counts of warning-bearing images.

4.3 Outputs

Typical outputs include:

- `detections` (bounding boxes + confidence)
 - `warnings` (e.g., low-confidence)
 - summary fields:
 - failures
 - retries
 - warning counts
 - totals
-

5. API Layer: FastAPI Server

5.1 Endpoints

- `/health` — readiness and detector status

- `/detect` — inference endpoint
- `/docs` — interactive documentation

5.2 Operational Behavior

- Uses a lifespan handler (avoids deprecated startup hooks).
 - Supports disabling model autoloader for tests via `SMART_MARINE_DISABLE_AUTOLOAD=1`.
-

6. Benchmarking

6.1 Script

- `scripts/benchmark_inference.py`

6.2 Metrics

- Throughput: FPS
- Latency distribution: p50 / p90 / p95 / p99
- Resource usage:
 - CPU snapshot (`psutil.cpu_percent`)
 - RSS memory sampling (`rss_before/after/peak`)

6.3 Stress Mode (time-based)

Supports running continuously for a time window:

- `--max-seconds <seconds>`
- RSS sampling cadence: `--rss-sample-every-n`
- records error rate:
 - `errors.count`
 - `errors.rate`

6.4 Performance Summary (CPU)

Observed CPU throughput in your environment:

- Webcam: ~18 FPS
- Video file: ~11 FPS

Interpretation:

- CPU is acceptable for demos and batch review.
 - Real-time (30+ FPS) requires GPU or optimized runtime.
-

7. Deployment

7.1 Docker

- Dockerfile provided for reproducibility.
- .dockerignore excludes model weights by default.

Run pattern:

- build image
 - mount weights
 - set SMART_MARINE_MODEL_PATH
-

8. QA / Testing

8.1 Test Suite

- pytest suite covers:
 - detector edge cases
 - batch empty directory behavior
 - API smoke tests
 - benchmark resilience tests (missing/invalid video)

8.2 CI/Developer Experience

- repo-root `pytest.ini` prevents collecting archive/backup scripts.
 - tests run from repo root.
-

9. Known Limitations & Production Caveats

9.1 Real-time feasibility

- CPU-only FPS is below 30 FPS target.
- GPU benchmarking is documented but may not be physically verified on every machine.

9.2 Long-duration operations

- Stress mode exists and supports time-based runs.
- Multi-hour drone ops require field validation.

9.3 Environment / Codec variability

- Video decoding performance depends on codec/container.
 - Network latency and camera behavior can affect real deployments.
-

10. How to Use (Operator Workflow)

10.1 Quick demo

- run API
- use /docs
- upload an image

10.2 Benchmarking

- run benchmark script on webcam/video
 - collect JSON report
-

12. Reproducibility Runbook (Exact Commands)

This section provides copy-paste commands to reproduce the core workflows: **API demo**, **benchmarking**, and **test suite**. All commands assume you are in the repository root.

12.1 Environment setup

Create virtual environment (if not already done)

```
python3 -m venv .venv
source .venv/bin/activate
```

Install runtime + dev dependencies

```
pip install -r smart_mairine_project/smart_marine_project/requirements.txt
pip install -r smart_mairine_project/smart_marine_project/requirements-dev.txt
```

12.2 Run the FastAPI server (demo mode)

```
cd smart_mairine_project/smart_marine_project
export SMART_MARINE_MODEL_PATH="models/ocean_waste_model_m2/weights/best.pt"
uvicorn api_server:app --host 0.0.0.0 --port 8000 --reload
```

- Open <http://localhost:8000/docs> for interactive API docs.
- Use /health to confirm the detector loaded.

12.3 Benchmark Webcam (CPU)

```
cd smart_mairine_project/smart_marine_project
python scripts/benchmark_inference.py --video 0 --max-frames 200 --rss-sample-every-n 50
```

- Results saved to `results/benchmark_*.json`.
- Expect ~18 FPS on CPU; check `fps` and `latency.p99`.

12.4 Benchmark Video File (CPU)

```
cd smart_mairine_project/smart_marine_project
python scripts/benchmark_inference.py --video path/to/your/video.mp4 --max-frames 200 --rss-
```

12.5 Stress Benchmark (10-minute time-based)

```
cd smart_mairine_project/smart_marine_project
python scripts/benchmark_inference.py --video 0 --max-seconds 600 --rss-sample-every-n 100
```

- Look for `errors.count`, `errors.rate`, and RSS trend in the JSON.

12.6 Run Test Suite

```
# From repository root
pytest -q
```

- Tests include edge cases, API smoke, and benchmark resilience.

12.7 Docker Build & Run (optional)

```
cd smart_mairine_project/smart_marine_project
docker build -t smart-marine .
docker run --rm -p 8000:8000 -v $(pwd)/models:/app/models:ro -e SMART_MARINE_MODEL_PATH=/app
```

13. API Examples (Requests & Responses)

Below are concrete examples of calling the `/detect` endpoint and the shape of the JSON response, including metadata and warnings.

13.1 Upload a single image (multipart/form-data)

```
curl -X POST "http://localhost:8000/detect" \
-F "file=@/path/to/test_image.jpg"
```

Response (pretty-printed)

```
{
  "run_id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "model_version": "best_pt_md5_abc123def",
  "timestamp": "2026-01-21T18:32:10.123Z",
  "detections": [
    {
      "class": "plastic",
      "confidence": 0.87,
      "bbox": {
        "x1": 120,
```

```

        "y1": 45,
        "x2": 210,
        "y2": 140
    }
}
],
"warnings": [],
"summary": {
    "total_images": 1,
    "detections": 1,
    "failures": 0,
    "retries": 0,
    "low_confidence_warnings": 0
}
}

```

13.2 Upload a directory of images (multipart/form-data)

```

curl -X POST "http://localhost:8000/detect" \
-F "files=@/path/to/img1.jpg" \
-F "files=@/path/to/img2.jpg"

```

Response (batch)

```

{
  "run_id": "3fa85f64-5717-4562-b3fc-2c963f66afa7",
  "model_version": "best_pt_md5_abc123def",
  "timestamp": "2026-01-21T18:33:05.456Z",
  "detections": [
    {
      "image": "img1.jpg",
      "objects": [
        {
          "class": "plastic",
          "confidence": 0.91,
          "bbox": { "x1": 30, "y1": 10, "x2": 95, "y2": 78 }
        }
      ]
    },
    {
      "image": "img2.jpg",
      "objects": []
    }
  ],
  "warnings": [
    "img2.jpg: low confidence detections (max 0.42)"
  ]
}

```



```
],
  "summary": {
    "total_images": 2,
    "detections": 1,
    "failures": 0,
    "retries": 0,
    "low_confidence_warnings": 1
  }
}
```

13.3 Health Check

```
curl -X GET "http://localhost:8000/health"
```

Response

```
{
  "status": "ok",
  "detector_loaded": true,
  "model_path": "/app/models/ocean_waste_model_m2/weights/best.pt",
  "model_version": "best_pt_md5_abc123def"
}
```

14. Appendix: Documentation Map

- README.md — entry point
- MODEL_DECISIONS.md — modeling rationale
- OBSERVABILITY.md — logging/metrics plan
- LIMITATIONS.md — operational caveats
- DEMO_MEDIA.md — demo assets
- TROUBLESHOOTING.md — common failures
- GPU_BENCHMARKING.md — CUDA guidance
- ONNX_BENCHMARKING.md — ONNX/TensorRT evaluation plan